

Jenkins CI/CD Automation Report

GitHub Integration, Pipeline Automation and Jenkins Fundamentals

1. Introduction

Jenkins is an open-source automation server used to automate software development activities such as building, testing and deploying applications. CI/CD is important because it helps teams deliver software faster and more reliably by automatically validating code changes and moving them through the delivery process.

2. CI/CD

CI (Continuous Integration) automatically builds and tests code whenever developers make changes. **CD (Continuous Delivery/Deployment)** automates the process of preparing or releasing validated software.

Developer → GitHub → Jenkins → Build → Test → Deployment

3. Jenkins Architecture

Component	Purpose
Jenkins Controller	Manages Jenkins configuration, jobs, scheduling and orchestration.
Jenkins Agents	Provide execution environments where builds and tests can run.
Jobs	Define automated tasks such as builds, tests and deployments.
Plugins	Extend Jenkins with integrations and additional functionality.
Executors	Allow Jenkins to run build tasks concurrently on available nodes.

4. Freestyle Jobs

Freestyle projects provide a graphical configuration approach for defining source-code management, build triggers, build steps and post-build actions. They are useful for simple automation tasks and for learning the Jenkins interface.

5. Declarative Pipeline

A Jenkins Pipeline defines the CI/CD workflow as code. A **Declarative Pipeline** uses a structured syntax that makes stages, agents and build steps easy to understand and maintain. The pipeline is commonly stored in a **Jenkinsfile** inside the source-code repository.

6. Jenkinsfile

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }

    stage('Build') {
      steps {
        // Install dependencies / build application
      }
    }

    stage('Test') {
      steps {
        // Run automated tests
      }
    }
  }
}
```

```
}  
}  
}  
}
```

7. GitHub Webhooks

A GitHub webhook can notify Jenkins immediately when code is pushed to the repository. Jenkins receives the event and can trigger the configured pipeline automatically.

GitHub Push → Webhook → Jenkins → Pipeline

8. Jenkins Credentials

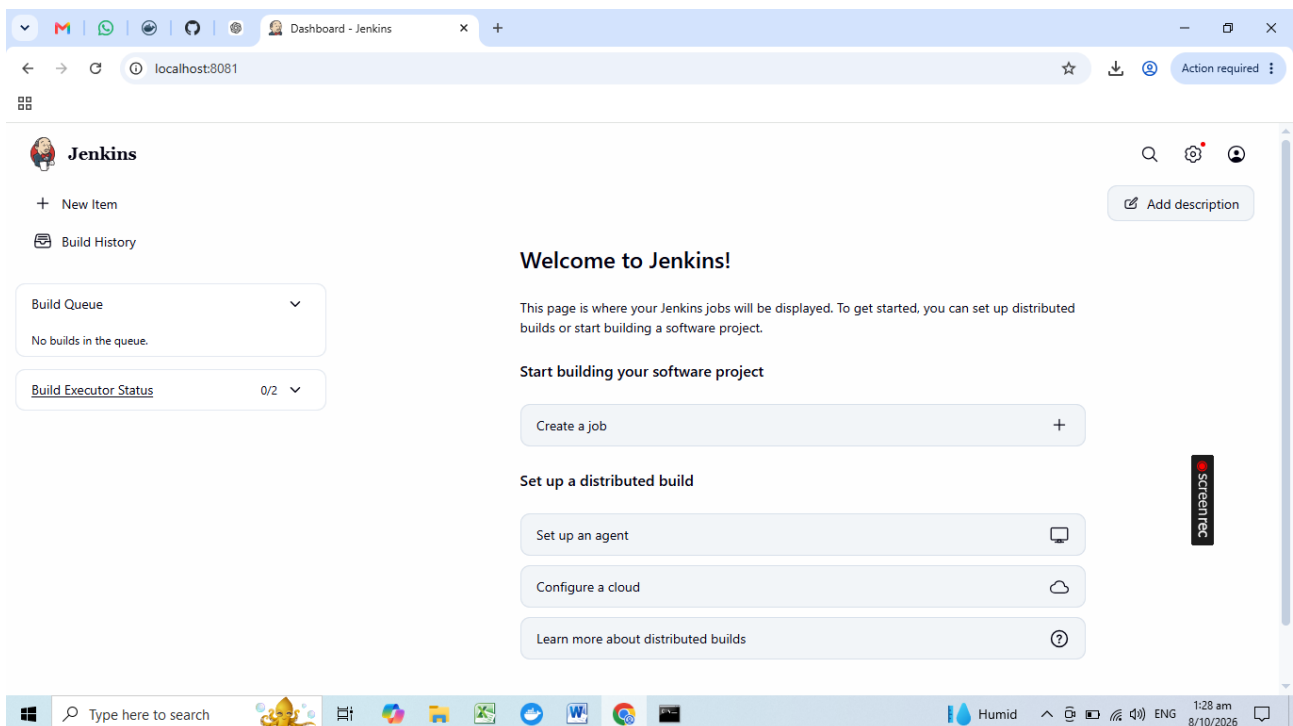
GitHub credentials, tokens and other secrets should be stored securely in **Jenkins Credentials**. They should not be written directly into the Jenkinsfile or exposed in build logs. The pipeline can reference the stored credential when authentication is required.

9. Pipeline Stages

Stage	Purpose
Checkout	Get source code from the configured repository.
Build	Install dependencies and/or build the application.
Test	Run automated tests and validate the application.

10. Screenshots

The following screenshots should be added as evidence of the completed Jenkins setup and execution. Replace each placeholder with the corresponding screenshot from your environment.



jenkins-Declarative-Pipeline #6

localhost:8081/job/jenkins-Declarative-Pipeline/6/console

Jenkins / jenkins-Declarative-Pipeline / #6

```
[Pipeline] bat
C:\ProgramData\Jenkins\.jenkins\workspace\jenkins-Declarative-Pipeline>echo Test completed successfully
Test completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins 2.580.1

#6 - jenkins-Declarative-Pipeline

localhost:8081/job/jenkins-Declarative-Pipeline/lastBuild/stages/

Jenkins / jenkins-Declarative-Pipeline / #6 / Stages

Started by Ankumpeta vimala Started 2 min 38 sec ago Took 10 sec Changes

Start Checkout SCM Checkout Build Test Post Actions End

Search

- Checkout SCM 2s
- Checkout 2s
- Build 0.72s
- Test 0.39s
- Post Actions 98ms

Post Actions 98ms Started 2m 29s ago Jenkins

Pipeline completed successfully! 41ms

Pipeline completed successfully!

Jenkins 2.580.1

Started by user Ankumpeta vimala
Obtained Jenkinsfile from git <https://github.com/ankumpetvimala/Jenkins-pipeline.git>
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\jenkins\workspace\jenkins-Declarative-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential github-credentials
> git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\jenkins\workspace\jenkins-Declarative-Pipeline\.git
timeout=10
Fetching changes from the remote Git repository
> git.exe config remote.origin.url <https://github.com/ankumpetvimala/Jenkins-pipeline.git> # timeout=10
Fetching upstream changes from <https://github.com/ankumpetvimala/Jenkins-pipeline.git>
> git.exe --version # timeout=10
> git --version # 'git version 2.55.0.windows.3'
using GIT_ASKPASS to set credentials GitHub credentials for Docker practical repository

[Pipeline] bat
C:\ProgramData\Jenkins\jenkins\workspace\jenkins-Declarative-Pipeline>echo Test completed successfully
Test completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

11. Conclusion

The application was successfully integrated with GitHub and automated using Jenkins. The CI/CD workflow demonstrates source-code checkout, build and test stages, with GitHub webhook integration enabling automated pipeline execution. Jenkins Credentials provide a secure method for handling authentication details.